

Warning: Do not delete slides.

This includes extra credit slides and any problems you do not complete.

All problems, including extra credit, must be assigned to a slide on Gradescope. Failure to follow this will result in a penalty.

CS 6476 Project 3

Venkata Aashutosh Aripirala

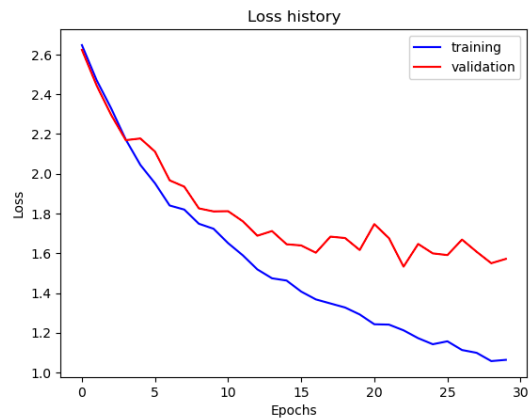
av84@gatech.edu

av84

904173725

Part 1: SimpleNet

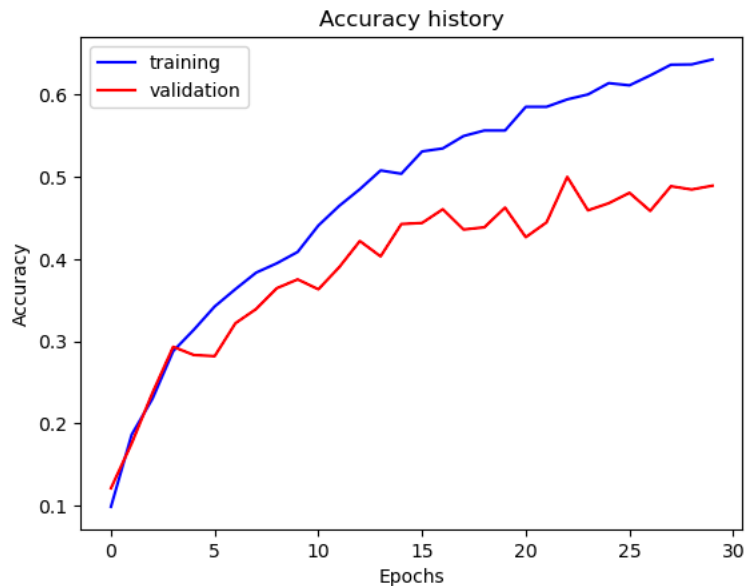
[Insert loss plot for SimpleNet here]



Final training accuracy: 64.288%

Final validation accuracy: 48.933%

[Insert accuracy plot for SimpleNet here]



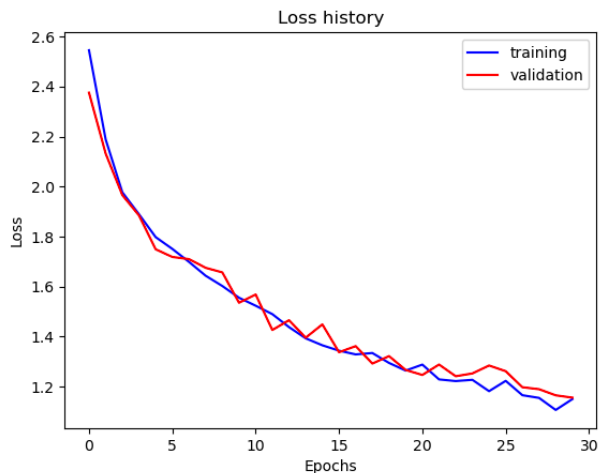
Part 2: SimpleNetFinal

Add each of the following (keeping the changes as you move to the next row):

	Training accuracy	Validation accuracy
SimpleNet	64.29%	48.93%
+ Jittering	58.19%	51.87%
+ Zero-centering & variance-normalization	74.67%	56.53%
+ Dropout regularization	67.44%	54.00%
+ Making network "deep"	61.31%	54.93%
+ Batch normalization	60.33%	61.2%

Part 2: SimpleNetFinal

[Insert loss plot for SimpleNetFinal here]



Final training accuracy: 60.335%

Final validation accuracy: 61.2%

[Insert accuracy plot for SimpleNetFinal here]



Part 2: SimpleNetFinal

[Name 10 different possible transformations for data augmentation.]

1. Random Horizontal Flip
2. Random Vertical Flip
3. Random rotation
4. Random Resizing crop
5. Color jitter
6. Random Affine Transform
7. Gaussian Blur
8. Random patch erasing
9. Random perspective warp
10. Gaussian Noise

[What is the desired variance after each layer? Why would that be helpful?]

The desired variance of the activations(outputs) is 1, along with a desired mean of 0. We implement this using BatchNorm. The batchnorm is crucial for mitigating risk of Internal Covariance Shift, which is the unstable change in layer distributions during training. This norm prevents that problem, allowing us to use faster learning rates, while avoiding the training process divergence. This allows for convergence to optimality quicker.

Part 2: SimpleNetFinal

[What distribution is dropout usually sampled from?]

Dropout is usually sampled from the Bernoulli distribution, as dropout can be compared to a Bernoulli trial, with a probability '1-p' of choice of dropping a neuron. The value "p" in `nn.Dropout(p)` refers to the probability of an element being dropped.

[How many parameters does your base SimpleNet model have? How many parameters does your SimpleNetFinal model have?]

SimpleNet: 56,895

SimpleNetFinal: 26,675

[What is the effect of batch norm after a conv layer with a bias?]

When batchnorm follows a conv layer + bias, the bias essentially becomes redundant, and this can be attributed to the following explanation.

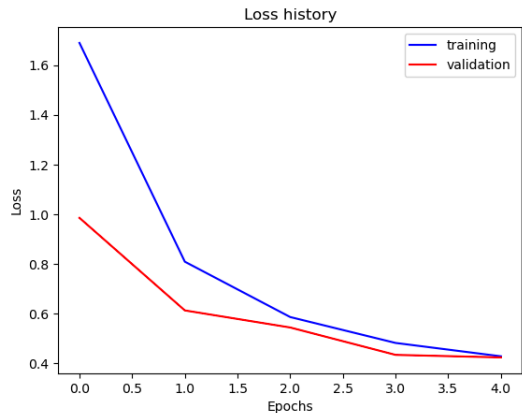
$$\text{Output} = \text{conv}(\text{input}) + \text{bias}$$

$$\text{Norm_output} = (\text{output} - \text{mean}) / \text{variance}$$

Since the constant bias is added to all elements, it is also present in the mean, and therefore is subtracted in the numerator of the `norm_output` equation.

Part 3: ResNet

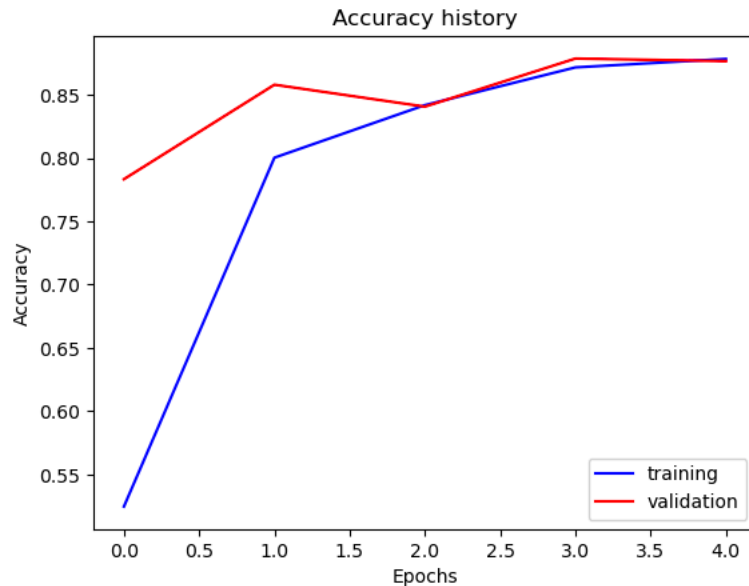
[Insert loss plot here]



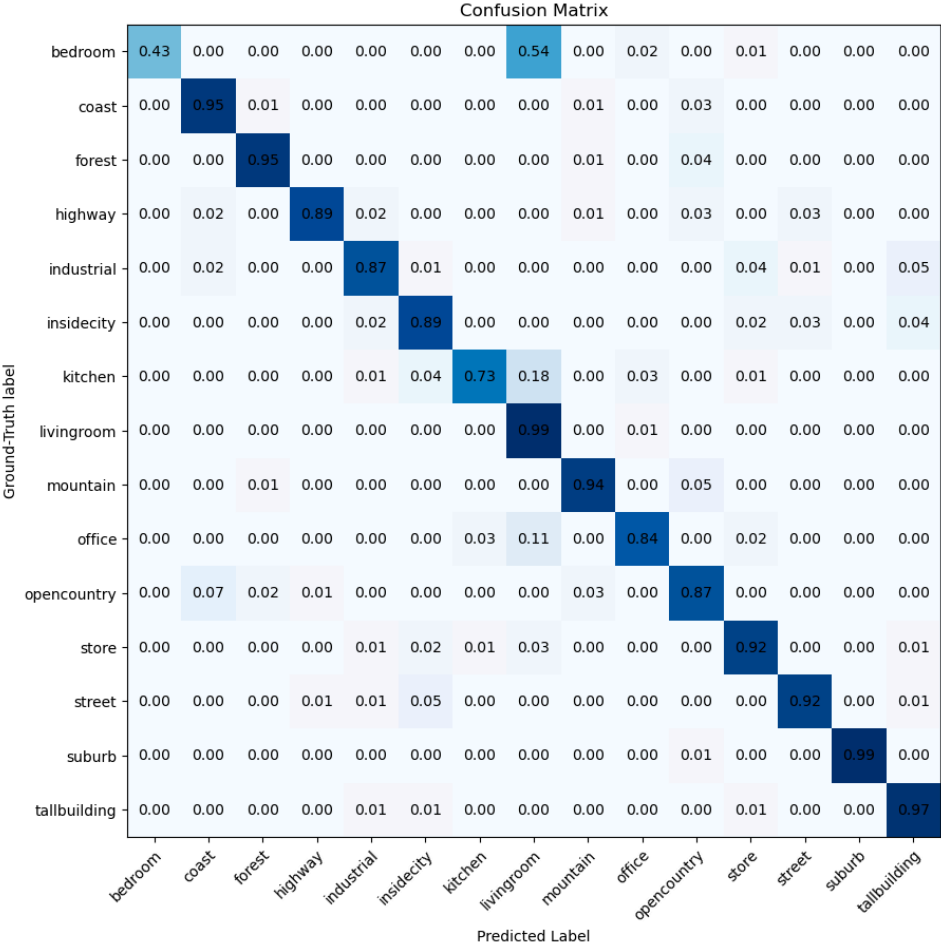
Final training accuracy: 87.8392%

Final validation accuracy: 87.667%

[Insert accuracy plot here]



Part 3: ResNet



Part 3: ResNet

This misclassification of predicting living room for what is actually a bedroom might've happened due to similarities in the visual and contextual space of the ResNet model. Both often contain similar kind of furniture, with common objects like chairs, tables, and lamps. From the model's perspective, the learned features of textures and shapes in both these classes are alike to an extent that it can struggle to find subtle characteristics that differentiate the two.

Misclassified: True Label 'bedroom', Predicted 'livingroom'

Image: image_0146.jpg



Image: image_0027.jpg



Image: image_0184.jpg



Part 3: ResNet

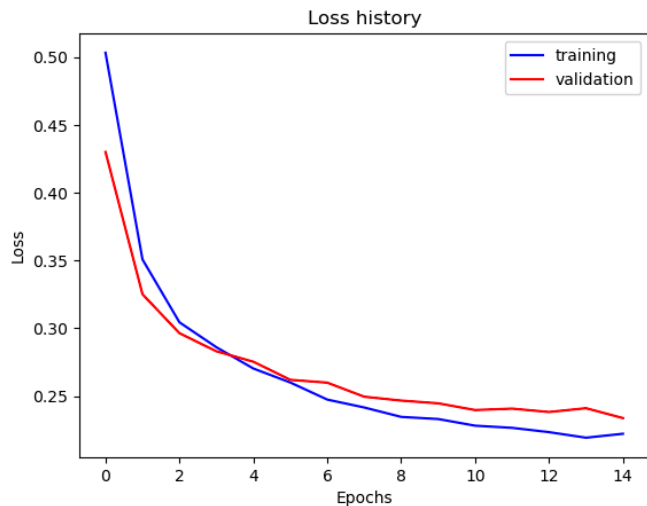
[What does fine-tuning a network mean?]

Fine-tuning is a transfer learning technique where we take a neural network that's already been trained on a large dataset, and adapt it for a new and specific task. This allows for us to train from a good and knowledgeable starting point, rather than having to train a model from scratch for every task. In summary, its re-training a pre-trained model on a smaller, more focused dataset to adapt its general knowledge to a specific task or domain, allowing a model to leverage its existing capabilities while becoming more accurate and specialized for a particular use case.

[Why do we want to "freeze" the conv layers and some of the linear layers from a pre-trained ResNet? Why can we do this?]

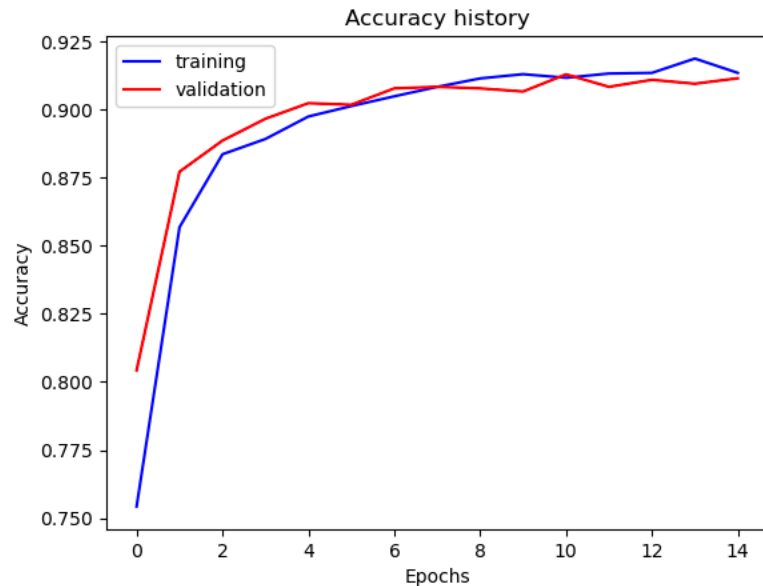
We freeze the conv layers of a pre-trained ResNet to preserve and leverage the understanding of the general visual knowledge already contained in the network. This allows us to reduce the number of trainable parameters, while also preventing over-fitting on small datasets. This strategy is also effective because we can transfer the understanding learnt by the early conv layers, like edges and textures, to our task. Therefore, we only need to train the final task-specific layers, to just learn how to combine the understandings from the early layers into our classification problem.

Part 4: Multi-label Scene Attributes (Extra Credit)



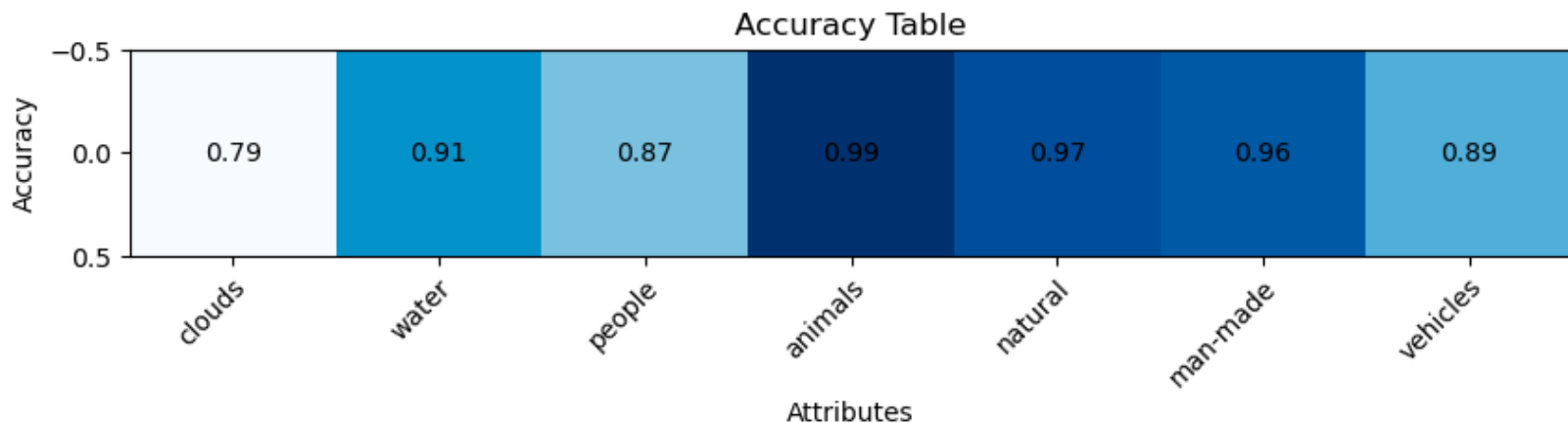
Final training accuracy: 91.3401%

Final validation accuracy: 91.1428%



Part 4: Multi-label Scene Attributes (Extra Credit)

[Insert visualization of accuracy table obtained from your final MultilabelResNet model.]



Part 4: Multi-label Scene Attributes (Extra Credit)

[List 3 changes that you made in the network compared to the one in part 3.]

The final fully connected layer of this network has 7 output neurons, for the 7 classes / scene attributes instead of 15 (in the ResNet from part 3). The loss function in this is BCEWithLogitsLoss, unlike the Resnet from part 3 which used CrossEntropy. We also explicitly use a Sigmoid activation function in Part 4, which is different from the usage of softmax in part 3.

[Is the loss function of the ResNet model from part 3 appropriate for this problem? Why or why not?]

The cross-entropy loss is used for Multi-class single label classification, where each input point belongs to one class. It effectively uses Softmax to force the probabilities to compete against each other to define one true label. In the case of Part 4, CrossEntropy would penalize the model for predicting high probabilities for multiple attributes, which is something we don't want. That's why BCEWithLogitsLoss is better, since it treats the 7 attribute predictions independently allowing any number of them to be true for a single input image.

Part 4: Multi-label Scene Attributes (Extra Credit)

[Explain a problem that one needs to be wary of with multilabel classification. HINT: consider the purpose of visualizing your results with the accuracy table. You might want to do some data exploration here.]

A major problem to be wary of with multi-label classification is dataset imbalance, where some attributes are far more frequent than others. Data exploration of our training set confirms this, showing that the 'natural' attribute appears in 833 images, while the 'animals' attribute is present in only 31. This imbalance can be deceptive because a model can achieve high accuracy on a rare attribute like 'animals' (99%) simply by learning to almost always predict its absence. A high overall validation accuracy, such as our 92%, is completely hiding this flaw, which is why the per-attribute accuracy table is essential for true performance analysis. The table also reveals that imbalance isn't the only factor, as our model struggled most with clouds (79%) despite it being a common attribute, indicating it was also visually challenging. Therefore, we must be wary of class imbalance because it can create a false sense of high performance, a problem that can only be diagnosed with detailed tools like the accuracy table.

Extra credit (optional)

[Discuss what extra credit you did and provide analyses.]